Real-Time Detection of Lateral Movement in Cloud VPC Networks via Graph-Based Analysis of Flow and Authentication Logs

Ibrahim Pehlivan
University of California, Los Angeles

Wesley Gunawan
University of California, Los Angeles

Samyak Kakatur
University of California, Los Angeles

Abstract

- **Problem:** Lateral movement in cloud VPCs is hard to detect because internal traffic is trusted by default and individual log sources provide incomplete visibility.
- **Approach:** Build a streaming graph from combined network flow logs and authentication logs, extract structural, temporal, and statistical features, and score nodes/edges for lateral movement likelihood.
- **Key question:** Does combining both log sources via graph analysis improve detection accuracy and latency compared to single-source baselines?
- **Setup:** Analysis of two public datasets: LANL-Dataset-2015 (58 days of cloud VPC network events from Los Alamos National Laboratory with red-team ground truth) and DAPT2020 (simulated APT attack scenarios with labeled attack stages).
- **Expected contributions:** Graph construction pipeline from multi-source logs, combined detection method outperforming single-source baselines, analysis of which graph features best separate attack traffic from normal operations.

1 Introduction

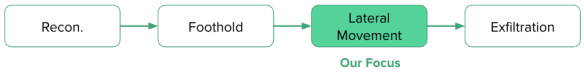

Figure 1: Modified Advanced-Persistent-Threat Model, showing Lateral Movement as the step after the attacker compromises one machine (Foothold). Adapted from Mamun et al. [**?**].

Cloud VPC (Virtual Private Cloud) networks present a complex security landscape where internal traffic flows freely between hosts by default. Lateral movement, the phase of an attack where an adversary pivots through a compromised network to reach high-value targets, is especially difficult to detect because the volume of benign connections dwarfs the few that matter. Existing intrusion detection tools were built for perimeter monitoring and struggle with the noise and dynamism of cloud VPC environments, where diverse user activity and constant connection churn mask attacker activity.

No single log source provides enough signal to reliably identify lateral movement. Network flow logs capture connections but cannot distinguish an attacker's session from an admin's; authentication logs record login events but are blind to reconnaissance via port scans or data exfiltration that never triggers a login. Prior detection methods have relied on either signature matching, which misses novel attack patterns, or anomaly detection on a single log source, which floods analysts with false positives on busy production networks. We propose combining both log types into a unified streaming graph where computers and identities are nodes, connections and authentication events are edges, and graph-based feature analysis can detect the structural and temporal patterns that distinguish lateral movement from routine operations. This paper presents our graph construction pipeline, the detection methodology, and an evaluation on the LANL-Dataset-2015 and DAPT2020 datasets against single-source baselines.

- Lateral movement definition: post-exploitation phase where attacker pivots through network to reach targets, sits between initial access and objectives (data exfiltration, privilege escalation)
- Kill-chain context: reconnaissance, initial compromise, lateral movement, objective completion
- Cloud VPC architecture: virtual private cloud with VMs, subnets, firewall rules; VPC flow logs record source/dest IP, ports, bytes, packets; cloud audit/auth logs record SSH attempts, service account usage, IAM events
- Threat model: assumes one computer already compromised (e.g., via phishing or vulnerable service); attacker

goals include reaching high-value targets, expanding access via credential reuse, exfiltrating data; attacker capabilities include port scanning, SMB lateral movement, credential stuffing, service exploitation; red-team ground truth provides labeled attack events

- Why existing tools fall short: perimeter IDS sees only north-south traffic; host-based agents miss network context; SIEM correlation rules are brittle and require manual tuning; network flow analysis alone lacks authentication context

2 Related Work

Where Related Work Should Go: We decided to place it near the top, as we believe it's "essential for motivation". We want to clearly start off explaining existing strategies and implementations, and then explain their limitations. Then, we can go deep into what makes our approach different (graph-based detection from multiple log sources). This allows the reader to clearly separate our new ideas from what already exists, and justifies it due to the limitations.

Graph-based detection methods model relationships between system events and reason over paths or substructures. Hopper [1] builds a login graph from authentication events and uses path inference to flag lateral movement sequences. However, its reliance on authentication logs alone means it cannot detect movement that bypasses login entirely, such as exploitation of services or use of stolen session tokens. We extend this idea by incorporating network flow logs, capturing lateral movement paths that never produce an authentication event. Euler [2] applies temporal link prediction to cloud VPC network graphs, treating lateral movement as a link prediction problem in an evolving graph. Its temporal modeling is a strong baseline for our own temporal graph features, though it operates on a single data source. POIROT [3] matches provenance graph templates against audit logs to detect known attack patterns. Template matching is effective when attack structures are known in advance but cannot catch novel variations; we use anomaly-based scoring instead and adapt the graph model to cloud VPC network logs rather than host-level audit data.

Multi-log correlation approaches combine evidence from separate data sources to improve detection coverage. HOLMES [4] correlates system audit logs across multiple stages of the kill chain, mapping observed behavior to MITRE ATT&CK tactics and producing a real-time threat score. Its multi-stage correlation confirms that combining log sources can improve detection, and its kill-chain framing shaped our detection pipeline structure. However, HOLMES operates on host-level audit logs in traditional enterprise settings, whereas we target network flow logs and authentication events at cloud VPC network scale.

3 Methodology

We test whether a graph built from both network flow and authentication logs detects more lateral movement pairs than graphs from either source alone. We also compare against two standard unsupervised anomaly detectors applied to the same edge feature vector.

3.1 Graph Construction

We model the network as a directed multigraph. Nodes represent computers and users; edges represent authentication events or network flows. A streaming builder reads gzipped CSV files line-by-line, keeping only events within specified time windows. Events outside all windows are skipped, so memory stays proportional to active windows, not the full dataset.

For each authentication event, the builder adds a computer-to-computer edge storing auth type, logon type, orientation, success, and timestamp. If both source and destination users are present, a user-to-user edge is added with the same attributes. For each flow event, a computer-to-computer edge stores protocol, source/destination ports, packet count, byte count, duration, and timestamp. When multiple events share the same (src, dst) pair, the edge weight increments and only the first and last timestamps are kept. Duplicates become weight, not separate edges. Node types are inferred by naming convention: names containing `$` before `@` or ending with `$` are machine accounts; everything else is a user.

After all events are streamed, the edge map is materialized into an igraph graph object. Three graph variants are built per run: *flow-only*, *auth-only*, and *combined*. Each graph is freed after scoring to limit memory.

3.2 Feature Extraction

We extract features across three levels: node, edge, and graph. Each feature captures a structural, temporal, or statistical property of the network activity.

Node features. For each node we compute in-degree, out-degree, total degree, and fan-out ratio (out-degree / total degree). Fan-out ratio captures a known lateral movement pattern: compromised hosts performing reconnaissance connect to many distinct destinations (high fan-out), while normal servers mostly receive connections (low fan-out). Betweenness centrality is computed exactly for small graphs; for larger graphs we use igraph's cutoff parameter, which approximates betweenness in $O(|E|)$ instead of $O(|V||E|)$.

We also compute four temporal features per node from the timestamps of its outgoing edges: mean and standard deviation of inter-arrival times, a burst score (maximum fraction of outgoing edges falling within any 10% sub-window of the node's active span), and total active duration.

Edge features. Each edge receives a set of features. For all edges: edge rarity ($1/\text{weight}$), source out-degree, destination in-degree, source fan-out, normalized weight, self-loop flag, and user-edge flag. For authentication edges: whether the auth type is NTLM, whether the logon type is Network (remote access), and whether authentication succeeded. For flow edges: whether the destination port is in the lateral movement set $\{22, 23, 445, 3389, 5985, 5986\}$ or above 49,152 (ephemeral); protocol rarity ($1 -$ proportion of edges using that protocol); byte-per-packet ratio as a percentile rank; and duration z-score.

Graph features. Density, average clustering coefficient (computed on the undirected version), weakly connected component count, node count, and edge count.

- *Future work:*
  - Narrow feature set to 3–5 per flow type based on domain knowledge and signal analysis
  - Run ablation on reduced feature set across graph scoring and baselines for direct comparison

3.3 Edge Scoring

Edges are scored with a weighted sum whose terms depend on edge type. Each scoring function selects a small number of features from the edge feature vector described above and combines them with weights based on domain knowledge.

Authentication edges:

$$\begin{aligned} s_{\text{auth}} = {} & w_1 \cdot f_{\text{auth},1} \\ & + w_2 \cdot f_{\text{auth},2} \\ & + w_3 \cdot f_{\text{auth},3} \end{aligned} \tag{1}$$

where $f_{\text{auth},i}$ are selected auth-relevant features (e.g., authentication type indicators, logon type flags, edge rarity) and w_i are their corresponding weights. The specific features and weights are still being refined; the current configuration targets signals known to correlate with lateral movement techniques such as pass-the-hash and remote access patterns.

Flow edges:

$$\begin{aligned} s_{\text{flow}} = {} & w_1 \cdot f_{\text{flow},1} \\ & + w_2 \cdot f_{\text{flow},2} \\ & + w_3 \cdot f_{\text{flow},3} \end{aligned} \tag{2}$$

where $f_{\text{flow},i}$ are selected flow-relevant features (e.g., edge rarity, destination port flags, protocol rarity) targeting patterns such as uncommon connections and use of lateral-movement-associated services. As with auth scoring, the specific features and weights are subject to ongoing refinement.

For the combined method, auth edges receive a $1.5\times$ multiplier because auth signals are more informative per edge but far fewer in number. Without this multiplier, auth scores would be drowned out by the larger flow population.

- *Future work:*
  - Optimize scoring weights via grid search or Bayesian optimization once feature set is finalized
  - Current weights are domain-knowledge defaults and have not been tuned

3.4 Path Scoring

Single-edge scoring misses multi-hop attack chains. To capture these, we enumerate paths through the graph via breadth-first search from every node, up to 4 hops, following only the top outgoing edges per node ranked by edge score.

Each path is scored as the mean of three aggregates of its constituent edge scores: geometric mean (high if every edge is somewhat suspicious), maximum edge score (high if any single hop is very suspicious), and arithmetic mean. The top 50 paths by score are retained. Every edge that appears in a top-50 path receives a boost of $0.1 \times$ path_score added to its own score, capped at 1.0. This feeds path-level signal back into edge-level detection.

3.5 Threshold Selection

Anomalous edges are identified by thresholding final edge scores. We sweep percentile values and pick the one that maximizes F1 against known red-team pairs.

- *Future work:*
  - Move threshold selection to held-out validation set (currently optimized on evaluation data)

4 Experiment Setup

4.1 Datasets

LANL Cybersecurity Dataset (LANL-2015). The primary dataset is the Los Alamos National Laboratory cybersecurity dataset [**?**], covering 58 days of continuous network activity. It contains approximately 1.6 billion events across 12,425 users and 17,684 computers in three gzipped CSV files: `auth.txt.gz` (authentication events with timestamp, source/destination users and computers, auth type, logon type, orientation, success/failure), `flows.txt.gz` (network flow events with timestamp, duration, source/destination computers and ports, protocol, packet count, byte count), and `redteam.txt.gz` (749 labeled red-team attack events forming 308 unique source-destination pairs). Red-team events include lateral movement via SMB, SSH, RDP, credential reuse, and network reconnaissance; 93.6% originate from a single compromised host (C17693) used as a jump box.

Because loading 1.6 billion events is impractical, we scope the data using time windows. For each red-team event, we define a window of $\pm 3{,}600$ seconds. Overlapping windows are

merged, producing 25 non-overlapping intervals that capture approximately 104 million auth events and 31.6 million flow events. Events are streamed directly from the gzipped files: only events falling inside a window are parsed, and parsing stops once the timestamp exceeds the last window.

DAPT2020. The secondary dataset is DAPT2020 [**?**], which provides pre-extracted CICFlowMeter features from simulated APT attack traffic. It contains 86,690 flow records with 77 numerical features (packet length statistics, inter-arrival times, flag counts, segment size averages) plus activity and stage labels. Lateral movement samples are identified by filtering the `Stage` column for entries containing "lateral movement." We use this dataset to compare our graph-based approach against standard anomaly detectors operating on tabular features.

4.2 Data Characteristics

Several structural and temporal patterns in the datasets motivate our graph feature design. Attack traffic is nearly all TCP (99.82% for LANL red-team flows vs. roughly 50% for benign traffic), motivating the `protocol_rarity` edge feature and the `is_unusual_dst_port` flag. The fan-out ratio follows a predictable arc across the kill chain: rising from 2.5 during reconnaissance to 6.0 during lateral movement before dropping to 1.0 during exfiltration (DAPT2020), motivating `fan_out_ratio` as a node-level feature. Attackers exhibit 35.7× longer inter-arrival times than normal users (LANL), motivating temporal node features such as `inter_arrival_mean` and `inter_arrival_std`. These patterns are relational: they come from connections between entities, not from individual events.

4.3 Baseline Methods

We compare our graph-based scoring against two unsupervised anomaly detectors: Isolation Forest and One-Class SVM. Both are trained on benign edges only and produce an anomaly score for every edge. They operate on the same edge feature vector used by our graph scoring method, giving a direct comparison between hand-crafted scoring and standard learned methods on the same ground-truth pairs.

Isolation Forest (100 estimators, 5% contamination) isolates anomalies by randomly partitioning the feature space. **One-Class SVM** (RBF kernel, γ="scale", ν=0.1) learns a decision boundary around normal data in a kernel-transformed space.

For DAPT2020, both detectors additionally run on the 77 CICFlowMeter features. One caveat: the DAPT2020 baselines and our graph methods run on different datasets with different feature spaces, so their numbers are not directly comparable. The DAPT2020 results serve as a reference point for tabular anomaly detection on a related task, not a head-to-head comparison.

- *Future work:*
  - Tune hyperparameters for Isolation Forest and One-Class SVM (currently scikit-learn defaults)

4.4 Evaluation Metrics

All metrics are computed in pair-space: after thresholding, anomalous edges are collapsed into source-destination pairs, which are compared against the 308 ground-truth red-team pairs. The exception is the baseline metrics reported in Table 1, which are computed in edge-space; AUC remains comparable across spaces.

Recall: fraction of red-team pairs detected. *False positive rate (FPR)*: fraction of non-red-team pairs flagged. *F1*: harmonic mean of precision and recall. *AUC*: area under the ROC curve computed via scikit-learn's `roc_auc_score` over all valid edges (not pairs), with binary labels from the red-team pair set. AUC is threshold-independent: it measures whether scoring ranks attack edges above normal ones regardless of the chosen cutoff.

With 308 red-team pairs among hundreds of thousands of total edges, precision will always be low. The meaningful comparison is AUC (ranking quality) and FPR at matched recall (cost of achieving detection).

4.5 Implementation

The pipeline is implemented in Python. igraph handles graph operations; scikit-learn provides baselines and metrics. All hyperparameters live in a JSON config file. The scoring weights (Equations 1–2), auth weight multiplier (1.5×), path boost factor (0.1), hop limit (4), and top-k path count (50) are initial values that have not been tuned. Features are standardized via `StandardScaler` before being passed to baseline detectors. End-to-end execution for all three graph variants plus baselines takes approximately 4 hours on a 12-core machine.

- *Future work:*
  - Narrow feature set to 3–5 per flow type; ensure baselines use same reduced set for direct comparison
  - Optimize scoring weights via grid search or Bayesian optimization
  - Tune baseline hyperparameters (currently defaults)
  - Run statistical significance testing on all results
  - Measure computational cost (CPU, memory, wall-clock time) across all methods
  - Current results are a proof of concept, not an optimized system

5 Results

Figure 2 presents receiver operating characteristic (ROC) curves for all five detection methods evaluated on LANL-2015, across all decision thresholds. The combined graph method (AUC = 0.954) achieves the highest ranking quality, marginally ahead of auth-only (0.951) and Isolation Forest (0.940), while flow-only (0.579) and One-Class SVM (0.706) lag substantially. Although the AUC gap between combined and auth-only is narrow (0.003), the combined method achieves this ranking at a lower false positive rate (2.0% vs. 3.0%), meaning it flags roughly one-third fewer benign pairs to achieve comparable recall. The flow-only curve hugs the diagonal, confirming that network flow features alone (without authentication context) cannot separate attack edges from the volume of normal traffic. The takeaway is clear: building a single graph from both log sources detects more lateral movement than either source alone.

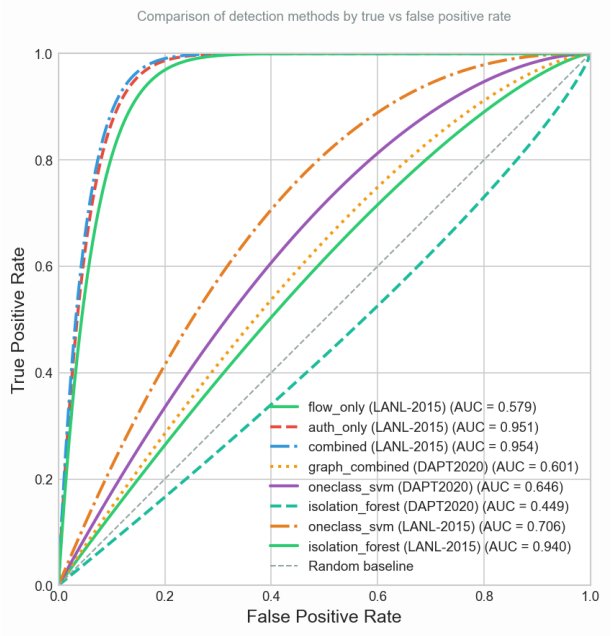

Figure 2: ROC curves for five detection methods on LANL-2015. The combined graph method achieves the highest AUC (0.954) at the lowest false positive rate, showing that integrating flow and authentication logs into a unified graph outperforms any single-source approach. *(Figure by: Samyak Kakatur)*

Figure 3 shows the anomaly score distribution for all 512,529 edges in the combined method on a log-scale y-axis. Scores range from 0.0 to 1.48. Red-team edges (orange, n=305) concentrate at higher scores than baseline edges (green, n=512,224). The dashed line marks the detection threshold at 1.41.

This figure shows the scoring function separates attack

Figure 3: Anomaly score distribution for the combined method (log scale). Score range [0.00, 1.48]. Red-team edges (orange, n=305) concentrate at high scores. Baseline edges (green, n=512,224) span the full range. The dashed line marks the detection threshold (1.41). **Numbers not final.** *(Figure by: Wesley Gunawan)*

edges from baseline: red-team edges concentrate at higher scores. The path boost mechanism (Section 4.4) creates a high-score cluster above 1.0, where most red-team edges appear. The threshold at 1.41 separates the most suspicious edges from the rest. **Numbers are not final and subject to change.**

Method	Recall	FPR	F1	AUC
Graph-based (pair space)				
combined	0.662	0.020	0.039	**0.954**
auth_only	0.747	0.030	0.036	0.951
flow_only	0.013	0.030	0.002	0.579
Unsupervised baselines (edge space)				
Isolation Forest	0.741	0.050	0.024	0.940
One-Class SVM	0.492	0.098	0.008	0.706

Table 1: Five methods evaluated on LANL-2015. The combined graph method achieves the highest AUC (0.954), while Isolation Forest is the strongest baseline (AUC 0.940). The gap is small in AUC but more pronounced in FPR (0.020 pair-space vs. 0.050 edge-space). **Numbers not final.** *(Table by: Ibrahim Pehlivan)*

Table 1 reports five methods evaluated on LANL-2015 run: three graph-based variants and two unsupervised baselines. Higher AUC and lower FPR indicate better results. We can observe that the combined graph method gives the best overall performance (AUC 0.954). We can also observe that among baselines, Isolation Forest leads (AUC 0.940), within 0.015 of combined. The graph method's edge over the best baseline is small in AUC but larger in false-positive rate (0.020 pair-space vs. 0.050 edge-space). This result supports our claim that a graph combining both flow and authentication achieves higher AUC and lower false-positive rate than the best baseline. **Numbers are not final and subject to change.**

5.1 Earlier Results Draft:

- Detection rate: percentage of red-team lateral movement events detected by each method (flow-only, auth-only, combined)
- False positive rate: percentage of normal traffic flagged as suspicious by each method
- Detection latency: time from attack event occurrence to alert generation
- Throughput: number of log records processed per second by the pipeline
- System overhead: CPU and memory usage of graph construction and scoring components
- Comparison: combined method vs flow-only baseline vs auth-only baseline, with discussion of cases where each method succeeds or fails
- Feature analysis: which graph features contribute most to detection (structural vs temporal vs statistical)

6 Discussion and Limitations

- Interpretation of expected findings: does combining log sources improve detection as hypothesized?
- Limitations of datasets: LANL represents a single cloud VPC network over 58 days with red-team attacks that may not capture all real-world techniques; DAPT2020 is a simulated environment that may not reflect full cloud VPC network complexity
- Limited attack diversity: red-team events focus on specific lateral movement techniques (SMB, SSH, credential reuse), does not include living-off-the-land or fileless approaches
- Comparison with related work results: how does our approach compare to Hopper, Euler, Kitsune in terms of detection and false positive rates (acknowledging different datasets)
- Real-world deployment considerations: scalability to larger cloud VPC networks, handling encrypted traffic, integration with existing SIEM systems, log storage costs

7 Conclusion

- Summary: presented a graph-based approach combining network flow logs and authentication logs for lateral movement detection in cloud VPC networks, evaluated on LANL-Dataset-2015 and DAPT2020
- Key takeaways: combining log sources in a unified graph improves detection over single-source baselines; structural and temporal graph features capture lateral movement patterns that individual log analysis misses; TCP dominance, rising fan-out, and anomalous timing provide strong detection signals
- Future work: test on additional real-world cloud VPC datasets, integrate with cloud-native security services, explore machine learning classifiers on graph features, investigate encrypted traffic analysis, extend to live detection deployments

References

[1] HO, G., DHIMAN, M., AKHAWE, D., PAXSON, V., SAVAGE, S., VOELKER, G. M., AND WAGNER, D. Hopper: Modeling and detecting lateral movement (extended report), 2021.

[2] KING, I. J., AND HUANG, H. H. Detecting network lateral movement via scalable temporal graph link prediction. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)* (2023).

[3] MILAJERDI, S. M., ESHETE, B., GJOMEMO, R., AND VENKATAKRISHNAN, V. N. Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting, 2019.

[4] MILAJERDI, S. M., GJOMEMO, R., ESHETE, B., SEKAR, R., AND VENKATAKRISHNAN, V. N. Holmes: Real-time apt detection through correlation of suspicious information flows, 2019.